

M14 Review / Repair - Teaching RC2

OPEN ONLY AFTER SAVED ATTEMPT

P01 repair - Batch vs streaming

Streaming is not "batch but faster". The source is logically unbounded and state/progress/failure handling is continuous.

Repair principle: Batch has a known/bounded input for one run/window.

Key proof: rerun batch -> replay offsets/events

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P02 repair - Events, brokers and topics

If topic/partition/offset are unclear, consumer groups and replay feel like magic.

Repair principle: An **event/message** has key/value plus metadata/headers/timestamp.

Key proof: offset -> position within one partition

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P03 repair - Producers and message keys

Producer key choice controls partition affinity/order for related events; ignoring delivery errors can lose messages silently.

Repair principle: Producer serializes key/value bytes and sends to topic partition.

Key proof: flush/callback -> delivery evidence

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P04 repair - Consumers and safe processing

A consumer can crash between processing and committing offset. Correct ordering determines whether data is lost or replayed.

Repair principle: Consumer subscribes/assigns partitions and polls records.

Key proof: commit before target then crash -> loss/skip risk

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P05 repair - Partitions and ordering

Kafka's scalability comes from partitions; ordering is normally guaranteed within a partition, not across partitions.

Repair principle: Each partition has its own monotonically increasing offsets/order.

Key proof: across partitions -> no global order

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P06 repair - Offsets, group position and replay

Replay and recovery depend on exactly which partition offsets were processed and committed.

Repair principle: Offsets identify records within a partition.

Key proof: seek/assign -> explicit replay position

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P07 repair - Consumer groups and scaling

Partitions define maximum parallelism for one consumer group at a moment. Extra consumers beyond partition count sit idle.

Repair principle: Consumers with same group.id cooperate; Kafka assigns partitions among them.

Key proof: 3 -> 5 -> ~2 idle

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P08 repair - Delivery semantics and idempotency

Delivery semantics are end-to-end. Kafka/client settings alone do not guarantee your database/API side effect occurs exactly once.

Repair principle: At-most-once may commit/progress before processing -> possible loss, fewer duplicates.

Key proof: idempotent sink -> duplicate delivery -> same durable effect

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P09 repair - Serialization, schemas and evolution

Streaming producers/consumers deploy independently; an uncoordinated field/type change can break continuously running consumers.

Repair principle: JSON is human-readable but still needs a schema/contract for required fields/types/meaning.

Key proof: new event_type -> consumer policy required

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P10 repair - Python producer and consumer workflow

The hands-on goal is to connect the conceptual log model to real client behavior without hiding offsets/keys/errors.

Repair principle: Start/verify Kafka training broker from Compose before rewriting client code.

Key proof: commit -> after processing

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.